Vectors: the Geometry of Data

Data becomes arrows; similarity becomes an angle

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

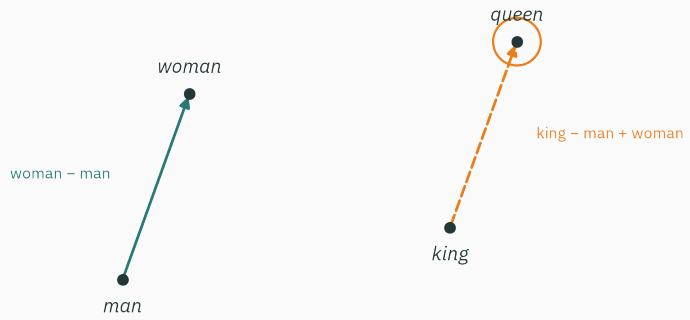
Word embeddings as vectors

Example 3 from Lecture 1: word-embedding arithmetic

V VISUAL

L2 explained Example 1 (the `nan` loss); Example 2 returns in L17. Today is Example 3:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$



a 2-D shadow of a 300-dimensional embedding space (→ L3)

Recall from L1: an **embedding** stores each word as a vector of learned numbers — "king" is a point in $\mathbb{R}^{50}$. The relation is then a claim about displacement and direction; we verify both numerically today.

A second example · “people like you”

Every streaming app claims to know “people like you”. But taste isn’t a number — what could “like you” even **mean**, mathematically?

Film	**You**	**Priya**	**Rohan**
Dangal	5	4	0
3 Idiots	4	5	1
Interstellar	1	0	5
Chak De! India	**?**	5	2

Who is more “like you” — Priya or Rohan? **By how much?** And should the app recommend you *Chak De!*? By the end of the lecture you will compute all three answers — with the same formula that handles king – man + woman.

Today’s one idea

A data point is a vector. Similarity between data points is geometry — ultimately, an angle.

The route, tool by tool:

Step	Tool	Use
length	norms $\|x\|$	how big, how far apart
angle	dot product, cosine	both questions above
shadow	projection	the germ of least squares & PCA
grid	span, basis	what coordinates even mean

Module 1 starts here: L3 vectors → L4 matrices (they **move** vectors) → L5 eigenvectors → L6 SVD & PCA.

Learning outcomes

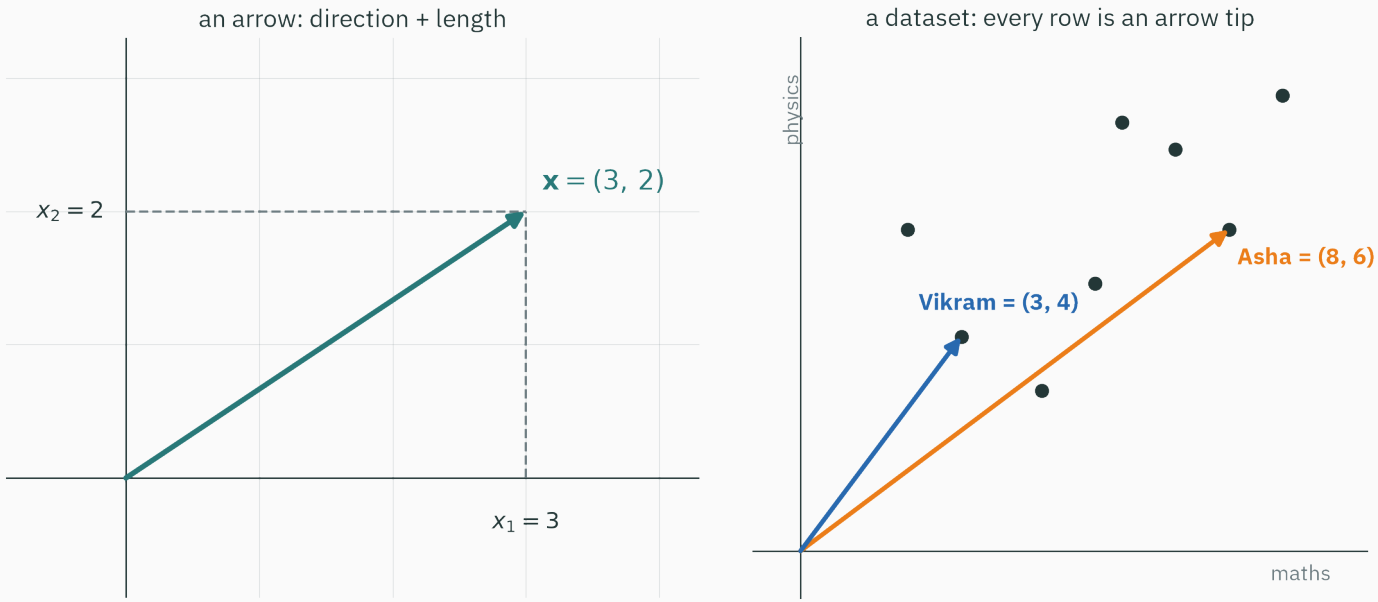
By the end of this lecture you will be able to:

1. Read a vector both ways: an **arrow** in space and a **row** of a dataset.
2. Measure size with **L1 / L2 norms** — and draw each norm's "unit ball".
3. Compute the **dot product** three ways: multiply–add, lengths × cos θ, shadow.
4. Derive $\cos\theta = \boldsymbol{x}^\top \boldsymbol{y} / (\|\boldsymbol{x}\|\|\boldsymbol{y}\|)$ from the school law of cosines (⭐).
5. Rank **real word embeddings** by cosine similarity and evaluate the opening analogy.
6. **Project** one vector onto another and verify the leftover is orthogonal.
7. Say when vectors **span** a line vs the plane, and what an (orthonormal) **basis** buys.

Vectors: arrows and data rows

One object, two readings

V VISUAL



Left: $(3, 2)$ is an **arrow** — a direction plus a length. Right: the same kind of pair is a **row of data** — Asha scored $(8, 6)$ in (maths, physics).

One object, two readings — an arrow you can draw, a row you can store. All of today is about switching between them at will.

Many AI objects can be represented this way

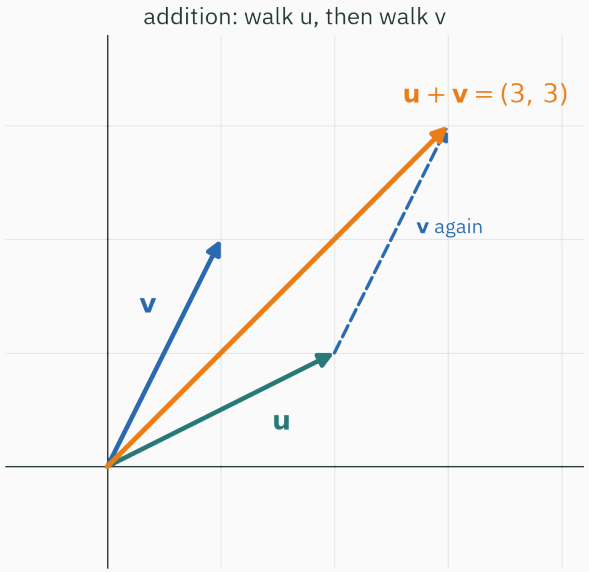
Object	The vector	Lives in
Asha's marks	$(8, 6)$	$\mathbb{R}^2$
your film ratings	$(5, 4, 1, \ldots)$	$\mathbb{R}^{\#\text{films}}$
the word "king" (GloVe)	50 learned numbers	$\mathbb{R}^{50}$
one MNIST digit	$28 \times 28 = 784$ pixel values	$\mathbb{R}^{784}$
3 s of speech at 16 kHz	48,000 samples	$\mathbb{R}^{48000}$

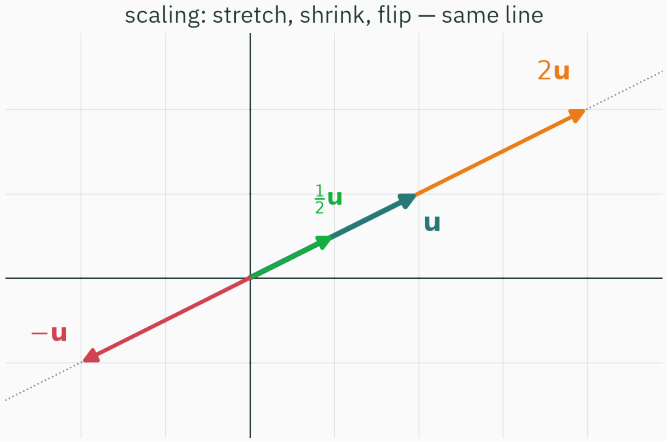
L2 callback: these entries are often stored as float32 or a lower-precision format — each coordinate then lives on an unevenly spaced number line.

From here on, "data" means: n points in $\mathbb{R}^d$ — n = how many, d = how rich.

Two moves only: add, and scale

V VISUAL





Addition: walk u, then walk v — tip to tail. Scaling: stretch, shrink, or flip along the same line.

These two moves are the whole meaning of “linear”. Everything in Module 1 — combinations, span, matrices — is built from exactly these.

The moves, in numbers and code

With $\boldsymbol{u} = (2, 1)$, $\boldsymbol{v} = (1, 2)$: everything happens slot by slot —

$$\boldsymbol{u} + \boldsymbol{v} = (3, 3) \qquad 2\boldsymbol{u} = (4, 2) \qquad -\boldsymbol{u} = (-2, -1)$$

```
u, v = np.array([2., 1.]), np.array([1., 2.])
u + v        # array([3., 3.])
2 * u        # array([4., 2.])
u * v        # array([2., 2.])  ← slot-by-slot, NOT the dot product
```

`u * v` multiplies slot-by-slot and keeps a **vector**. The product that carries geometry — `u @ v` — collapses two vectors into one number. It gets its own section, two sections from now.

The notation contract

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \in \mathbb{R}^d \qquad \boldsymbol{x}^\top = (x_1, \ldots, x_d)$$

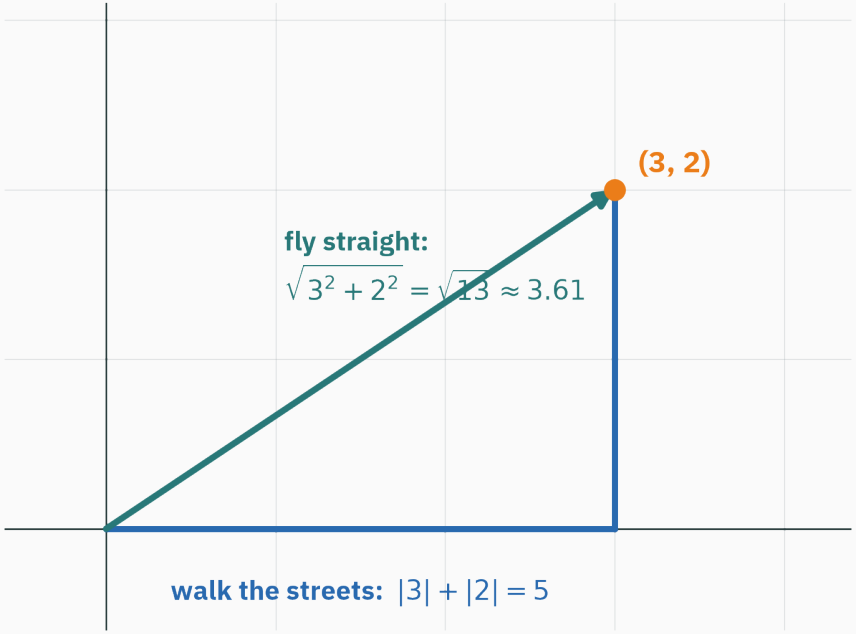
- **bold lowercase** $\boldsymbol{x}$: vectors (columns by default) · *italic* x_i: scalars & components
- CAPS A, X: matrices — from L4 on; the dataset is $X \in \mathbb{R}^{n \times d}$, one point per row
- n data points, d dimensions; an unadorned $\|\boldsymbol{x}\|$ always means the L2 norm

This is the same contract as MML and the CS229 linear-algebra handout — decks, tutorials, and assignments all speak it. Learn it once, read everything.

Norms: how big is a vector?

Two honest ways to measure one trip

V VISUAL



Walk the streets: $|3| + |2| = 5$ blocks. Fly straight: $\sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.61$.

Both are legitimate “sizes” of the same vector — they answer different questions. Let’s formalize both.

The L2 norm · as the crow flies

$$\|\boldsymbol{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2} = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}}$$

For the trip: $\|(3, 2)\|_2 = \sqrt{13} \approx 3.61$. Works in any dimension — for $(1, -2, 2) \in \mathbb{R}^3$: $\sqrt{1 + 4 + 4} = 3$.

- This is **the** default: $\|\boldsymbol{x}\|$ with no subscript means L2.
- It is Pythagoras, applied $d - 1$ times — flight distance in $\mathbb{R}^d$.

Keep the inner form $\sqrt{\boldsymbol{x}^\top \boldsymbol{x}}$ in view: **the length is secretly a product of x with itself** — that observation runs the next section.

The L1 norm · street by street

$$\|\boldsymbol{x}\|_1 = |x_1| + |x_2| + \cdots + |x_d|$$

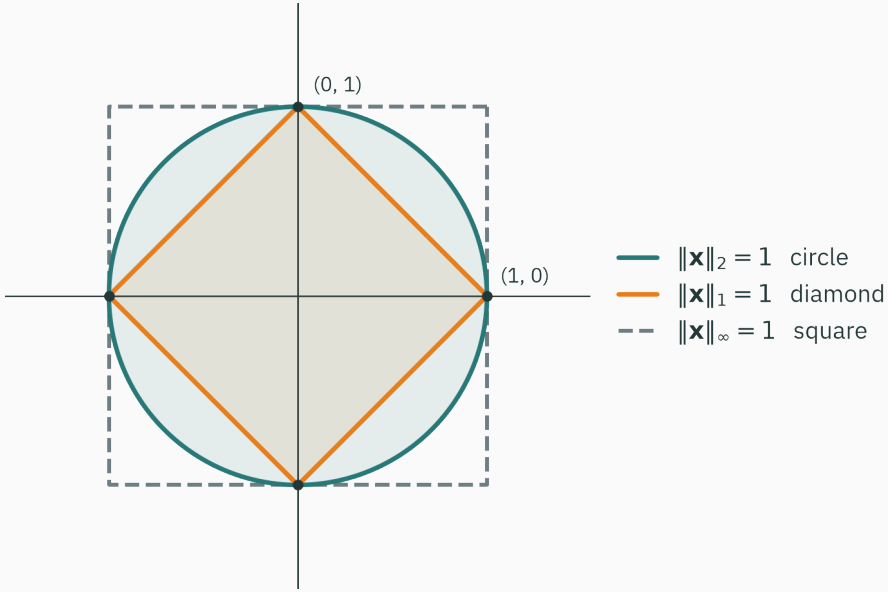
Worked example — a model's errors on four students, $\boldsymbol{e} = (2, -1, 3, -4)$:

Norm	Value	What it judges
$\|\boldsymbol{e}\|_1$	$2 + 1 + 3 + 4 = 10$	total miss — every error counts equally
$\|\boldsymbol{e}\|_2$	$\sqrt{30} \approx 5.48$	squares amplify — big misses dominate
$\|\boldsymbol{e}\|_\infty$	$\max = 4$	only the single worst case

One model, three verdicts. **Choosing a norm is choosing what you care about.**

Every norm draws a different “circle”

V VISUAL



All three earn the name “norm” by the same three axioms (MML 3.1): positive unless $\boldsymbol{x} = \mathbf{0}$, $\|c\boldsymbol{x}\| = |c|\,\|\boldsymbol{x}\|$, triangle inequality.

The L1 diamond has **corners** on the axes — the geometric clue for why L1 regularization can set weights exactly to zero. (ES 335 develops that result.)

Unit vectors · direction, distilled

$$\hat{\boldsymbol{x}} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|} \qquad \text{same direction, length exactly } 1$$

Numbers: $\boldsymbol{x} = (3, 2)$: $\hat{\boldsymbol{x}} = (3, 2)/\sqrt{13} \approx (0.832, 0.555)$ — check: $0.832^2 + 0.555^2 \approx 1$ ✓

Every nonzero vector splits cleanly: $\boldsymbol{x} = \|\boldsymbol{x}\| \cdot \hat{\boldsymbol{x}}$ — **how much** × **which way**.

Keep this distinction: cosine similarity will use direction and discard magnitude.

Distance · a norm in disguise

$$d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$$

Asha $(8, 6)$ vs Vikram $(3, 4)$: difference $(5, 2)$ → flight distance $\sqrt{29} \approx 5.39$, street distance 7.

```
asha, vikram = np.array([8., 6.]), np.array([3., 4.])
np.linalg.norm(asha - vikram)        # 5.385...  (L2)
np.linalg.norm(asha - vikram, 1)     # 7.0       (L1)
```

Every “nearest” in ML — nearest neighbour, nearest cluster centre, nearest embedding — is this one line of code plus a choice of norm.

One dot product, three readings

Reading 1 · multiply matching slots, then add

$$\boldsymbol{x}^\top \boldsymbol{y} = x_1 y_1 + x_2 y_2 + \cdots + x_d y_d = \sum_{i=1}^{d} x_i y_i$$

The running pair for the rest of this lecture: $\boldsymbol{x} = (3, 2)$, $\boldsymbol{y} = (4, 2)$:

$$\boldsymbol{x}^\top \boldsymbol{y} = 3 \cdot 4 + 2 \cdot 2 = 16$$

Two vectors in, **one number** out. But what does "16" mean? The next two readings answer that.

One meaning is already ours — pair $\boldsymbol{x}$ with itself: $\boldsymbol{x}^\top \boldsymbol{x} = 9 + 4 = 13 = \|\boldsymbol{x}\|^2$.

$\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}}$ **— the length was a dot product all along.**

Reading 2 · lengths × alignment

The claim (to be **earned** next section — it is today's ⭐):

$$x^\top y = \|x\|\,\|y\|\cos\theta$$

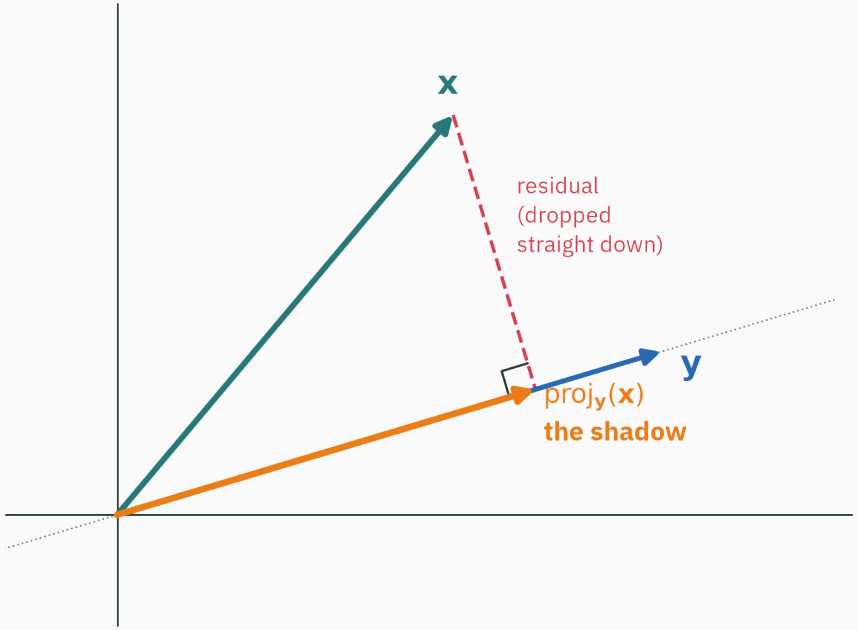
Check it against the running pair: $\|x\|\|y\| = \sqrt{13}\sqrt{20} \approx 16.12$, so

$$\cos\theta = \frac{16}{16.12} \approx 0.992 \quad \Rightarrow \quad \theta \approx 7.1^\circ \quad \text{— two nearly parallel arrows.}$$

Note what just happened: multiply–add on raw coordinates determined an angle, though nothing geometric was ever measured. That claim needs proof — the next section derives it.

Reading 3 · the shadow

V VISUAL



Dot x against a **unit** direction: $x^\top \hat{y} = \|x\| \cos\theta$ = the signed **length of x's shadow** on y's line. Running pair: $16/\sqrt{20} \approx 3.58$.

Read it as: "how much of x points the y way."

The alignment detector

Angle	$\cos\theta$	$\boldsymbol{x}^\top \boldsymbol{y}$	**Interpretation**
acute ($\theta < 90°$)	$+$	$+$	the vectors agree
right ($\theta = 90°$)	0	0	**orthogonal** — $\boldsymbol{x} \perp \boldsymbol{y}$, zero signed projection
obtuse ($\theta > 90°$)	$-$	$-$	they oppose

Check: $(3, 2)^\top (-2, 3) = -6 + 6 = 0$ → perpendicular. The axes too: $\boldsymbol{e}_1^\top \boldsymbol{e}_2 = 0$.

Orthogonal directions have zero signed projection onto each other. This is a geometric statement; it does **not** by itself imply that two measured features are statistically independent. Orthogonality returns twice today: residuals, and the nicest bases.

Three readings, one number

```
x, y = np.array([3., 2.]), np.array([4., 2.])
x @ y                                  # 16.0    reading 1: multiply-add
nx, ny = np.linalg.norm(x), np.linalg.norm(y)
theta = np.arccos(x @ y / (nx * ny))
np.degrees(theta)                      # 7.125   the angle between them
nx * np.cos(theta)                     # 3.578   reading 3: shadow length
(x @ y) / ny                           # 3.578   same shadow, no trig needed
```

Plant for L4: a matrix multiply is nothing but these, en masse — every entry of Wx is one dot product $w_i^\top x$, one "how much of x lies along my direction?" question.

Checkpoint: the perpendicular detector

Q QUESTION

$x = (3, 2)$. **Which y is orthogonal to x?**

A. $y = (2, 3)$

B. $y = (-2, 3)$

C. $y = (3, -2)$

D. $y = (-3, -2)$

Answer: the perpendicular detector

A ANSWER

Correct: B — $(-2, 3)$, since $3 \cdot (-2) + 2 \cdot 3 = 0$

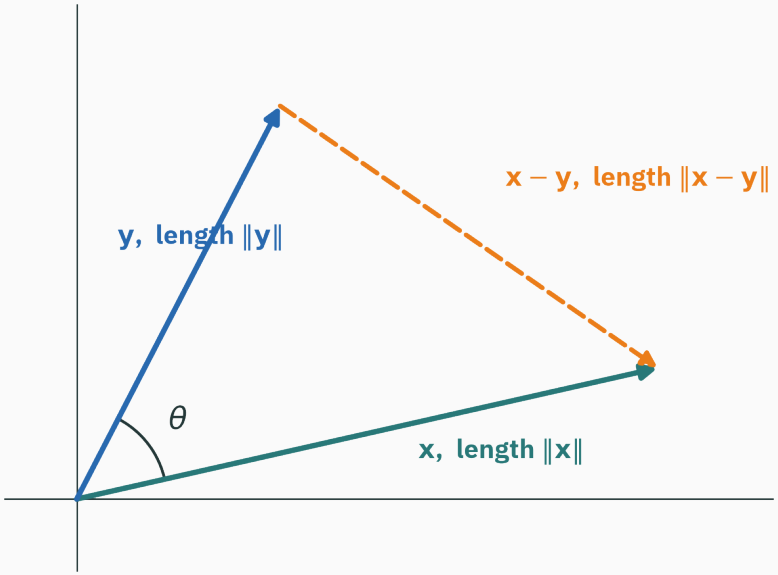
Why: in 2-D, "flip the slots, negate one": $(-x_2, x_1)$ is always $\perp (x_1, x_2)$. A gives 12 and C gives 5 (acute); D gives -13 — **opposing**, but only exactly zero means perpendicular.

Why multiply–add measures angle

Two currencies, one triangle

D DERIVATION

Claim (⭐): for any two nonzero vectors, $x^\top y = \|x\|\|y\| \cos\theta$.



The two sides of the claim speak different languages — coordinates vs geometry. The bridge: compute the third side $\|x - y\|^2$ **twice**, once in each language, and compare.

Step 1 · algebra expands the third side

D DERIVATION

$$\|\boldsymbol{x}-\boldsymbol{y}\|^2 = (\boldsymbol{x}-\boldsymbol{y})^\top(\boldsymbol{x}-\boldsymbol{y}) \qquad (\text{a norm}^2 \text{ is a self-dot})$$

$$= \boldsymbol{x}^\top\boldsymbol{x} - \boldsymbol{x}^\top\boldsymbol{y} - \boldsymbol{y}^\top\boldsymbol{x} + \boldsymbol{y}^\top\boldsymbol{y} \qquad (\text{expand the brackets})$$

$$= \|\boldsymbol{x}\|^2 + \|\boldsymbol{y}\|^2 - 2\,\boldsymbol{x}^\top\boldsymbol{y} \qquad (\text{since } \boldsymbol{y}^\top\boldsymbol{x} = \boldsymbol{x}^\top\boldsymbol{y})$$

Only two facts were used — brackets distribute, and order doesn't matter — and both read straight off the multiply–add formula: check $\boldsymbol{x}^\top(\boldsymbol{y}+\boldsymbol{z}) = \sum_i x_i(y_i + z_i) = \boldsymbol{x}^\top\boldsymbol{y} + \boldsymbol{x}^\top\boldsymbol{z}$.

Step 2 · school geometry measures the same side

D DERIVATION

The **law of cosines** (Class 10): any triangle with sides a, b, c and angle γ opposite c obeys

$$c^2 = a^2 + b^2 - 2ab\cos\gamma \qquad (\text{Pythagoras, plus a tilt correction})$$

Our triangle has sides $a = \|\boldsymbol{x}\|$, $b = \|\boldsymbol{y}\|$, $c = \|\boldsymbol{x} - \boldsymbol{y}\|$, and $\gamma = \theta$:

$$\|\boldsymbol{x} - \boldsymbol{y}\|^2 = \|\boldsymbol{x}\|^2 + \|\boldsymbol{y}\|^2 - 2\|\boldsymbol{x}\|\|\boldsymbol{y}\|\cos\theta$$

Sanity check the correction: at $\theta = 90°$ the cosine dies and pure Pythagoras remains. ✓

Step 3 · equate, cancel, done

D DERIVATION

Both computations measured the **same number** $\|x - y\|^2$:

$$\|x\|^2 + \|y\|^2 - 2\,x^\top y = \|x\|^2 + \|y\|^2 - 2\|x\|\|y\|\cos\theta$$

Cancel the squared lengths, divide by -2:

$$x^\top y = \|x\|\|y\|\cos\theta \quad\Longleftrightarrow\quad \cos\theta = \frac{x^\top y}{\|x\|\,\|y\|}$$

Check on the running pair: $16 = 16.12 \times 0.992$ ✓ ($\theta \approx 7.1°$, as promised).

Free corollary: $|\cos\theta| \le 1$ forces $|x^\top y| \le \|x\|\|y\|$ — that is the **Cauchy–Schwarz inequality**.

Cosine similarity: geometry becomes meaning

Angles in $\mathbb{R}^{300}$ · the formula becomes the definition

In $\mathbb{R}^2$ we **saw** θ and proved the formula. In $\mathbb{R}^{300}$ nobody sees anything — so flip the logic and let the formula **define** the angle:

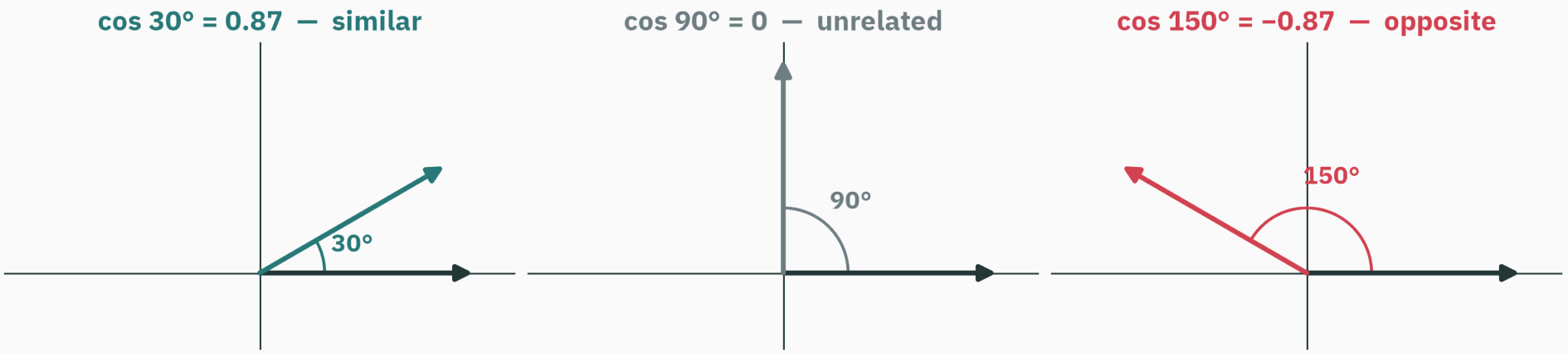
$$\text{cos\_sim}(\boldsymbol{x}, \boldsymbol{y}) := \frac{\boldsymbol{x}^\top \boldsymbol{y}}{\|\boldsymbol{x}\|\|\boldsymbol{y}\|} = \hat{\boldsymbol{x}}^\top \hat{\boldsymbol{y}} \qquad \text{— a dot product of unit vectors.}$$

This is legal and safe: Cauchy–Schwarz keeps it inside $[-1, 1]$, and any two non-collinear vectors span an ordinary 2-D plane where it **is** the visible angle. Collinear vectors give the endpoint angles $0°$ or $180°$.

Cosine is scale-invariant: $\text{cos\_sim}(10\boldsymbol{x}, \boldsymbol{y}) = \text{cos\_sim}(\boldsymbol{x}, \boldsymbol{y})$**. Lengths cancel — only directions are compared.**

Cosine similarity at three angles

V VISUAL



$+1$: same direction · 0: perpendicular · -1: opposite direction. For data, cosine can be a useful similarity score, but zero cosine does not automatically mean statistical or semantic independence.

“People like you”, by hand

Ratings on the three films everyone has seen: you $= (5, 4, 1)$, Priya $= (4, 5, 0)$, Rohan $= (0, 1, 5)$.

$$\text{you} \cdot \text{Priya}: \quad \frac{5 \cdot 4 + 4 \cdot 5 + 1 \cdot 0}{\sqrt{42}\sqrt{41}} = \frac{40}{41.5} \approx 0.96 \quad \Rightarrow \quad \theta \approx 15^\circ$$

$$\text{you} \cdot \text{Rohan}: \quad \frac{0 + 4 + 5}{\sqrt{42}\sqrt{26}} = \frac{9}{33.0} \approx 0.27 \quad \Rightarrow \quad \theta \approx 74^\circ$$

In this toy representation, a smaller angle means a more similar rating direction. Priya is therefore the nearer neighbour, and her 5★ rating suggests *Chak De!*.

What the toy recommender omits

The cosine calculation isolated one geometric step. A real system must also:

- distinguish “not rated” from an actual zero or low rating,
- account for users who rate everything high or low and films that are broadly popular,
- learn from many neighbours or fit latent user/item vectors rather than use three raw ratings.

The example is useful for learning cosine geometry; it is not a complete recommendation algorithm.

A word is 50 floats

GloVe (2014): read 6 billion words of text; assign every word a vector so that words appearing in **similar contexts** get similar vectors. No dictionary, no grammar — just co-occurrence.

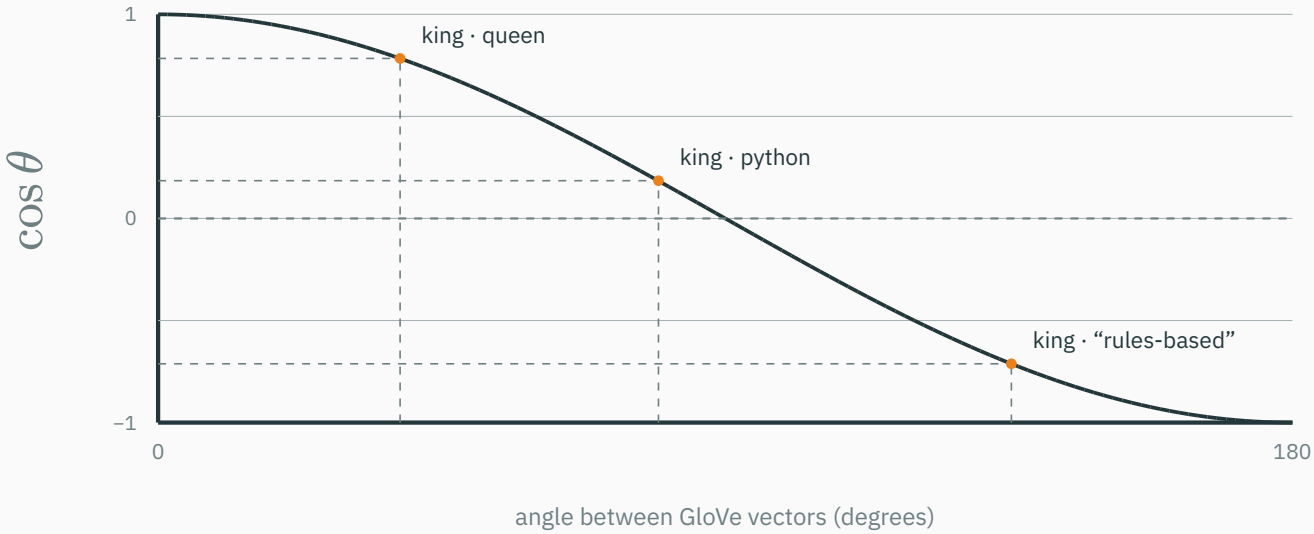
```
import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-50")   # 400k words × 50 floats
glove["king"][:6]
# array([ 0.5045,  0.6861, -0.5952, -0.0228,  0.6005, -0.135 ])

def cos_sim(u, v):
    return u @ v / (np.linalg.norm(u) * np.linalg.norm(v))
cos_sim(glove["king"], glove["queen"])       # 0.784  →  θ ≈ 38.4°
```

Coordinate 17 of “king” is usually not interpretable on its own. The useful signal lies mainly in relations among the 50 coordinates — **the geometry between word vectors**.

Cosine similarity for real word embeddings

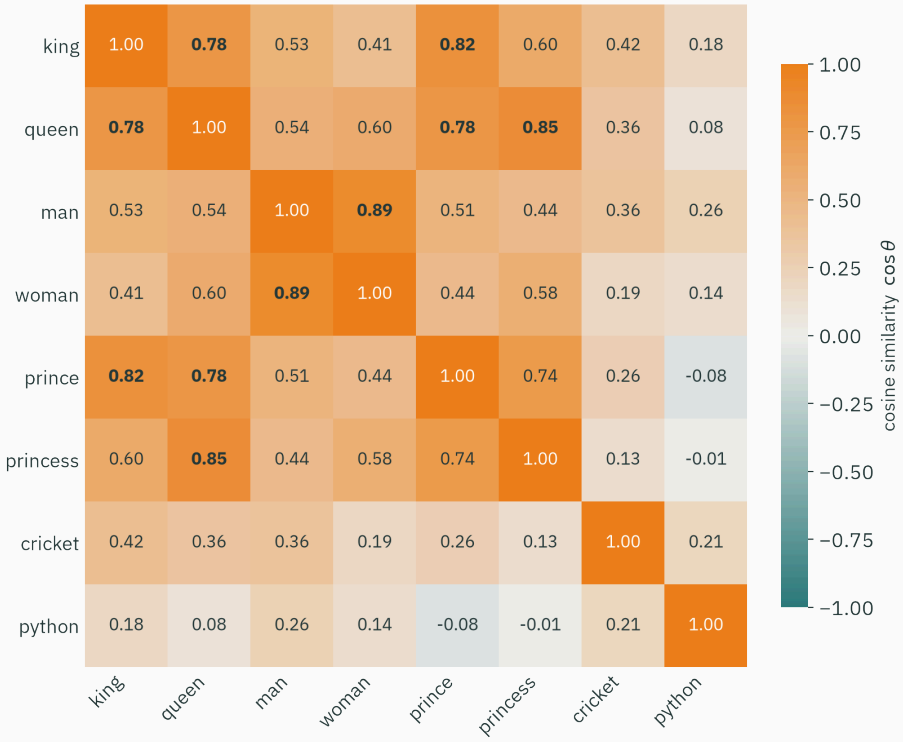
V VISUAL



More readings: uncle · aunt lands at $40.3°$, king · cricket at $65.1°$. Semantic neighbours sit at small angles; strangers drift toward $90°$ and beyond.

All 28 pairs at once

V VISUAL



Blocks are the point: a royal cluster (king–prince 0.82, queen–princess 0.85), a tight man–woman pair (0.89) — and python cold to all, even **opposed** to prince (-0.08).

Evaluate the word analogy

The bet behind the arithmetic:

$$\boldsymbol{v}_{\text{king}} - \boldsymbol{v}_{\text{man}} + \boldsymbol{v}_{\text{woman}} = \boldsymbol{v}_{\text{king}} + \underbrace{(\boldsymbol{v}_{\text{woman}} - \boldsymbol{v}_{\text{man}})}_{\text{a learned gender shift}}$$

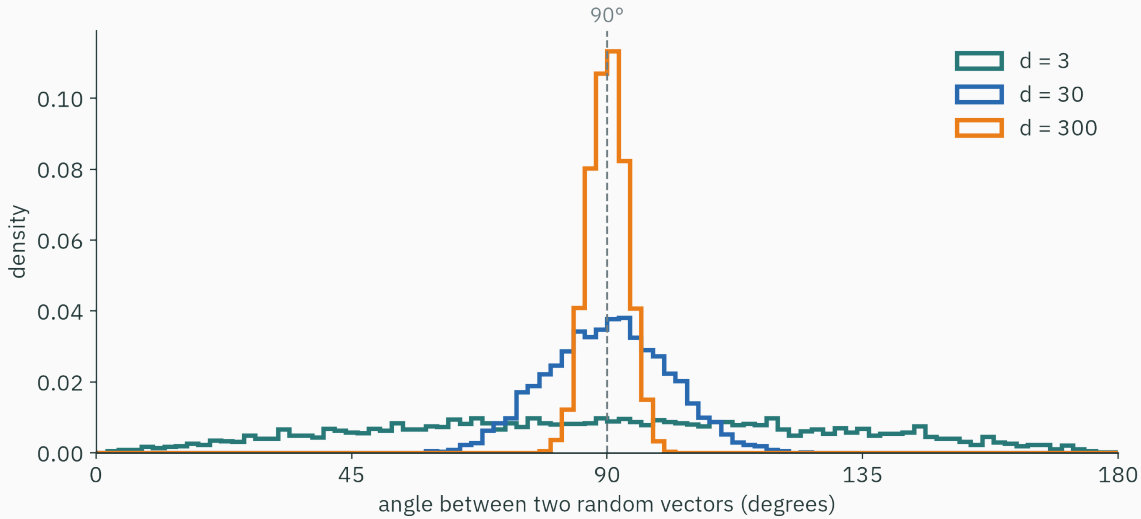
```
glove.most_similar(positive=["king", "woman"], negative=["man"])[0]
# ('queen', 0.852)  ← nearest of all 400k words, by cosine
```

The subtraction never "understood" gender. **Directions in this space consistently encode relationships** — man→woman and king→queen are nearly parallel arrows.

Honest small print: `most_similar` excludes the query words; keep them in and **king itself** outranks queen. The parallelogram is real but approximate — hence "≈", never "=".

Random high-dimensional directions are nearly orthogonal

V VISUAL



For independent isotropic random vectors, pairs in $\mathbb{R}^3$ cover a wide range of angles. In $\mathbb{R}^{300}$, their angles concentrate near $90°$.

So king · queen at $38°$ is far from this random baseline. Learned embedding clouds need not be isotropic, so the honest comparison is with angles from the same embedding model, not with a universal $90°$ rule.

Projections: shadows and residuals

From shadow length to shadow vector

Reading 3 gave a **number** — the shadow's length $\boldsymbol{x}^\top \hat{\boldsymbol{y}}$. The shadow itself is that length, aimed along $\hat{\boldsymbol{y}}$:

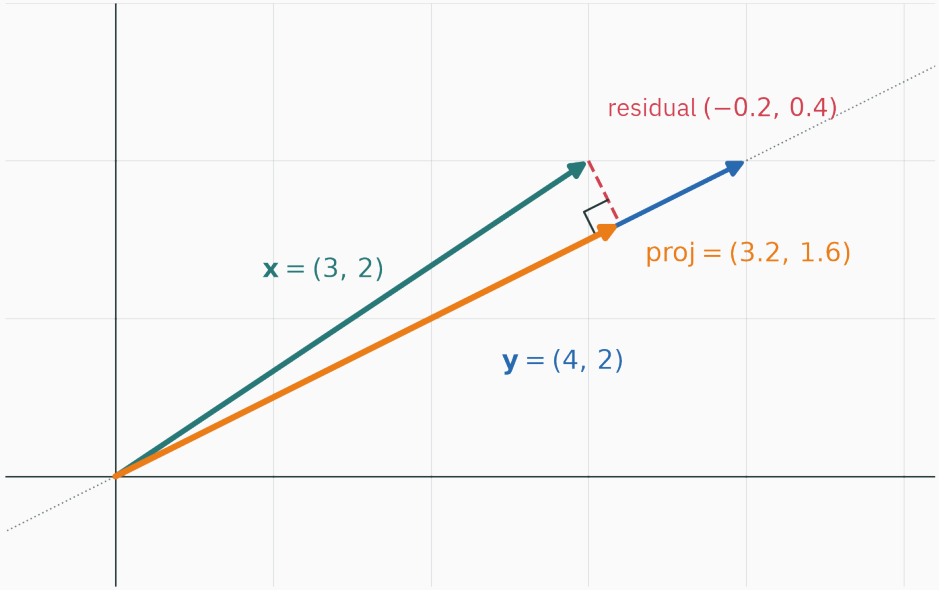
$$\text{proj}_{\boldsymbol{y}}(\boldsymbol{x}) = (\boldsymbol{x}^\top \hat{\boldsymbol{y}})\,\hat{\boldsymbol{y}} = \frac{\boldsymbol{x}^\top \boldsymbol{y}}{\boldsymbol{y}^\top \boldsymbol{y}}\,\boldsymbol{y}$$

(The right form just substitutes $\hat{\boldsymbol{y}} = \boldsymbol{y}/\|\boldsymbol{y}\|$ twice and uses $\|\boldsymbol{y}\|^2 = \boldsymbol{y}^\top \boldsymbol{y}$.)

Read it: **the multiple of $\boldsymbol{y}$ that best imitates $\boldsymbol{x}$**. Whatever's left — the residual $\boldsymbol{x} - \text{proj}$ — is the part of $\boldsymbol{x}$ that $\boldsymbol{y}$ cannot explain.

The shadow, in numbers you can check

V VISUAL



$$\frac{\boldsymbol{x}^\top \boldsymbol{y}}{\boldsymbol{y}^\top \boldsymbol{y}} = \frac{16}{20} = 0.8 \quad \Rightarrow \quad \text{proj} = 0.8\,(4, 2) = (3.2, 1.6)$$

Residual: $\boldsymbol{x} - \text{proj} = (-0.2, 0.4)$ — and $(-0.2, 0.4)^\top (4, 2) = -0.8 + 0.8 = 0$. Perpendicular, **exactly**.

Why perpendicular means “closest”

Slide a candidate point along y’s line and connect it to x: the connector is shortest where it meets the line at a right angle — any other landing point makes it a hypotenuse over that leg.

$\text{proj}_y(x)$ **is the best approximation of x on y’s line — and “best” is certified by an orthogonal residual.**

Widen “y’s line” to a whole plane of allowed directions and this same right-angle picture becomes **least squares** (L16) and **PCA** (L6). A surprising amount of “learning” is projecting.

Projection in three lines

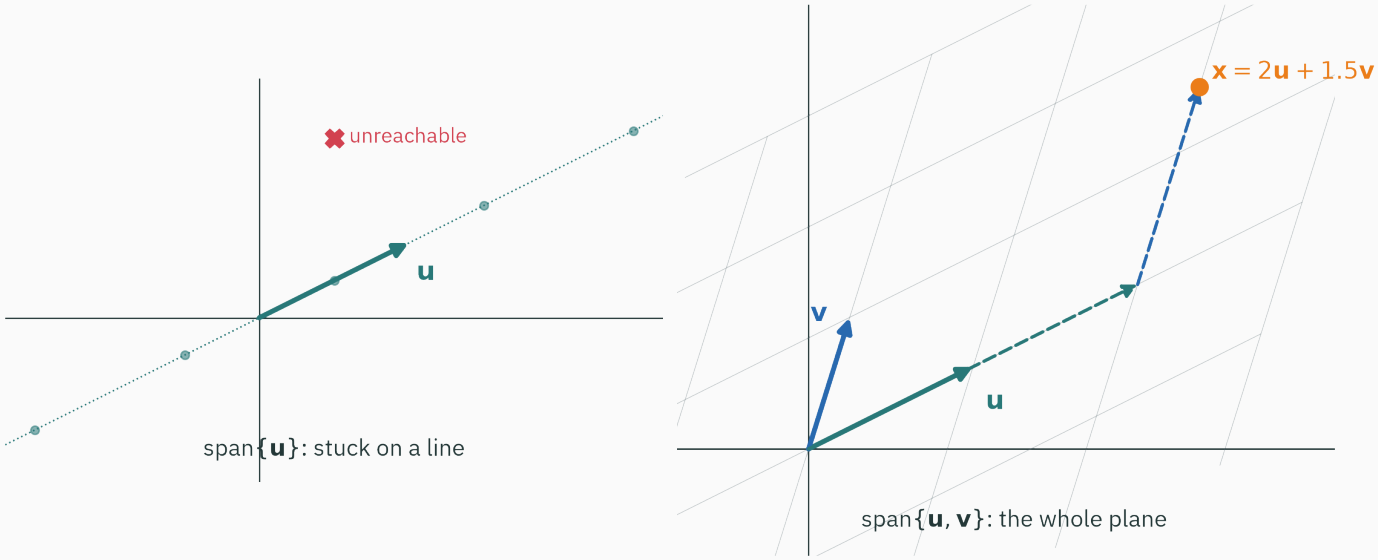
```
x, y = np.array([3., 2.]), np.array([4., 2.])
proj = (x @ y) / (y @ y) * y     # array([3.2, 1.6])
r = x - proj                     # array([-0.2,  0.4])
r @ y                            # 0.0 — the residual is ⊥, always
```

The zero is not luck; it is the definition doing its job. You will reuse exactly these lines inside least squares (L16) and PCA (L6).

Basis and span: the grid is a choice

Span · where can your moves take you?

V VISUAL



$\mathrm{span}\{\boldsymbol{u}\} = \{\alpha \boldsymbol{u}\}$: scaling alone keeps you on a **line**. $\mathrm{span}\{\boldsymbol{u}, \boldsymbol{v}\} = \{\alpha \boldsymbol{u} + \beta \boldsymbol{v}\}$: with two genuinely different directions, walk $\boldsymbol{u}$'s then $\boldsymbol{v}$'s and **every** point of the plane is reachable.

When a second vector adds nothing

Take $\boldsymbol{u} = (2, 1)$ and $\boldsymbol{v} = (4, 2) = 2\boldsymbol{u}$:

$\alpha\boldsymbol{u} + \beta\boldsymbol{v} = (\alpha + 2\beta)\,\boldsymbol{u}$ — every 'combination' is still a multiple of $\boldsymbol{u}$.

Linearly independent = no vector in the set is a combination of the others.
Dependent = a redundant direction; the span doesn't grow.

Data version: a feature `total = maths + physics` adds a **column** to your table but **no new direction** to the data. That redundancy is why matrices have rank (L4) and why data compresses (L6).

Drag a span yourself

I INTERACTIVE

INTERACTIVE

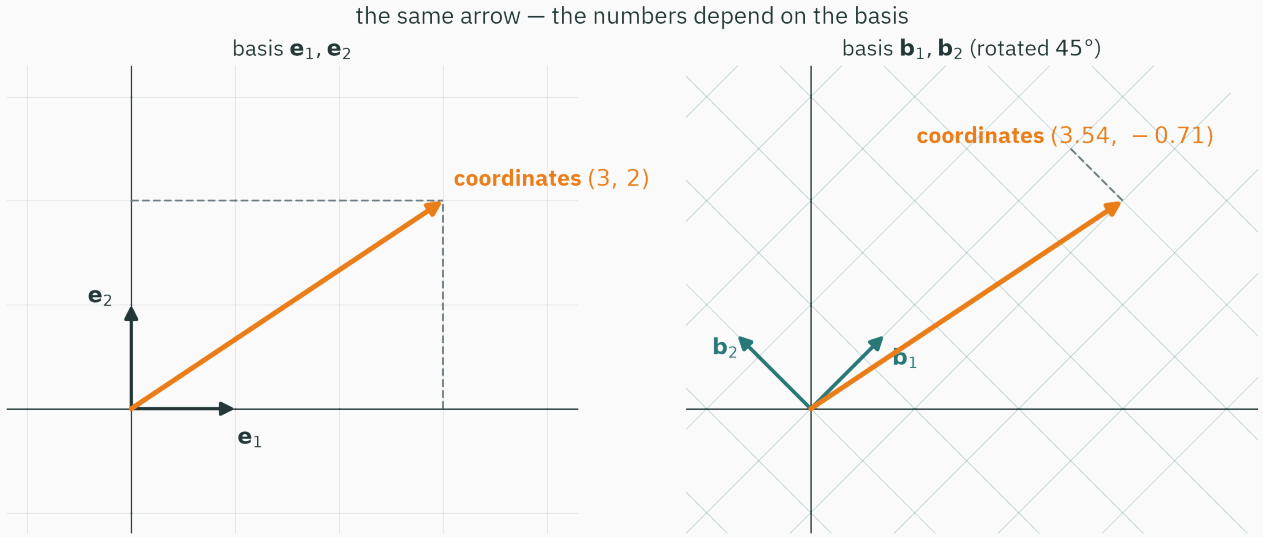
↗ textbooks.math.gatech.edu/ila/spans.html

Interactive Linear Algebra (GaTech) — in the span demos, drag $\boldsymbol{u}$ and $\boldsymbol{v}$ and watch $\mathrm{span}\{\boldsymbol{u}, \boldsymbol{v}\}$ collapse from the whole plane to a line the instant the arrows align.

Two minutes of dragging beats twenty static slides — do it tonight. This free textbook is the reference explorable for all of Module 1 (L3–L6).

A basis is a chosen grid

V VISUAL



A **basis** = a set of vectors that is (1) independent and (2) spans the space.
Consequence: every vector gets **exactly one address** — its coordinates.

Same arrow, two grids, two addresses: $(3, 2)$ vs $(3.54, -0.71)$. **The arrow is real; the numbers are an artifact of the grid you chose.**

Coordinates in an orthonormal basis

If $\boldsymbol{b}_1, \boldsymbol{b}_2$ are **unit length** and **mutually perpendicular** (orthonormal), addresses come free — each coordinate is a projection:

$$\boldsymbol{x} = (\boldsymbol{x}^\top \boldsymbol{b}_1)\, \boldsymbol{b}_1 + (\boldsymbol{x}^\top \boldsymbol{b}_2)\, \boldsymbol{b}_2$$

Check against the figure's rotated grid — $\boldsymbol{b}_1 = (1,1)/\sqrt{2}$, $\boldsymbol{b}_2 = (-1,1)/\sqrt{2}$, $\boldsymbol{x} = (3,2)$: $\boldsymbol{x}^\top \boldsymbol{b}_1 = 5/\sqrt{2} \approx 3.54$ and $\boldsymbol{x}^\top \boldsymbol{b}_2 = -1/\sqrt{2} \approx -0.71$ ✓ — the figure's numbers.

In an orthonormal basis, coordinates are projections: reading an address costs d dot products.

L6 teaser: PCA = let the data cloud elect its own favourite orthonormal grid.

Checkpoint: four quick calls

Q QUESTION

Commit to all four before the reveal.

A. $\cos\theta$ between x and y is 0.8. Your teammate rescales $x \to 10x$. New cosine?

B. $\mathrm{span}\{(2,1),(4,2)\}$: a point, a line, or the whole plane?

C. $\mathrm{proj}_{y}(y) = ?$

D. Can **three** vectors in $\mathbb{R}^2$ be linearly independent?

Answers: four quick calls

A ANSWER

A. 0.8 — unchanged. The 10 scales $\boldsymbol{x}^\top \boldsymbol{y}$ and $\|\boldsymbol{x}\|$ alike and cancels. Cosine sees direction only.

B. A line — $(4, 2) = 2 \cdot (2, 1)$: the second move is the first in disguise.

C. $\boldsymbol{y}$ itself — the coefficient is $\boldsymbol{y}^\top \boldsymbol{y} / \boldsymbol{y}^\top \boldsymbol{y} = 1$: a vector is its own shadow on its own line.

D. No — two independent arrows already span $\mathbb{R}^2$; any third is a combination of them. “How many independent directions fit” is exactly what **dimension** means.

Practice problems

Try on paper; verify in NumPy.

1. For $\boldsymbol{x} = (1, -2, 2)$: compute $\|\boldsymbol{x}\|_1, \|\boldsymbol{x}\|_2, \|\boldsymbol{x}\|_\infty$. Which is biggest — and is that always so?
2. A new user rates $(1, 1, 5)$. Cosine with you $= (5, 4, 1)$? More or less "like you" than Rohan's 0.27?
3. Find t so that $(3, 2) \perp (4, t)$.
4. Project $\boldsymbol{x} = (1, 4)$ onto $\boldsymbol{y} = (2, 0)$; write $\boldsymbol{x}$ = shadow + residual and verify the residual is $\perp \boldsymbol{y}$.
5. Show $(1, 1) \perp (1, -1)$, normalize both, and write $(3, 2)$ in this basis via two dot products.
6. In the heatmap, king · cricket ≈ 0.42 but prince · python ≈ -0.08. Give one geometric interpretation of each number.

The reading map

Source	What it gives you
MML Ch 2.1–2.4	vectors and vector spaces — today's arrows, done carefully
MML Ch 3.1–3.3	norms, inner products, distances — today's rulers
CS229 linear-algebra notes	the course's reference handout; keep it at hand through L6
3b1b *Essence of Linear Algebra* ch 1–3	today in moving pictures; chapter 3 is the perfect L4 pre-watch

All free: `mml-book.github.io` · `cs229.stanford.edu/section/cs229-linalg.pdf` · the 3b1b playlist on YouTube. Quiz questions come from lecture beats, and every beat above has a book home.

Lecture 3 — summary

- **A vector is an arrow AND a data row**; a dataset is n arrows in $\mathbb{R}^d$ (each entry a float32 — L2).
- **Norms measure size**: L2 flight, L1 streets, L∞ worst case — unit balls: circle, diamond, square.
- **One dot product, three readings**: $\sum_i x_i y_i = \|\boldsymbol{x}\|\|\boldsymbol{y}\| \cos\theta$ = shadow × length.
- ⭐ The **law of cosines** turns multiply–add into an angle meter — and **defines** angles in $\mathbb{R}^{300}$.
- **Cosine similarity** ranks words, films, and people; `king − man + woman ≈ queen` compares approximately parallel displacement vectors.
- **Projection** = closest point on a line; the residual is orthogonal — the germ of least squares and PCA.
- **Span** is where your moves reach; a **basis** is the grid coordinates are written in; orthonormal bases read coordinates by dot products.

Similarity is an angle.

Next: **Matrices as Linear Maps** — a dense neural-network layer uses a matrix to transform its input vector.